

Codeforces Round 1113 (Div. 2)

A. You Delete, I Delete

1 second, 256 megabytes

Alice and Bob are given a binary string s of length n . It contains at least one 0 and at least one 1.

They each perform exactly **one** operation in the following order:

- First, Alice chooses an occurrence of 0 in s and deletes it.
- Then, Bob chooses an occurrence of 1 in the resulting string and deletes it.

Alice wants the final string to be lexicographically[†] as large as possible, while Bob wants it to be lexicographically as small as possible. Determine the final string if both players act optimally.

^{*}A binary string is a string consisting only of the characters 0 and 1.

[†]For two distinct binary strings a and b of the same length, a is lexicographically smaller than b if, at the first position where they differ, a has the smaller digit.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 100$). The description of the test cases follows.

The only line of each test case contains a binary string s of length n ($3 \leq n \leq 100$).

It is guaranteed that s contains at least one 0 and at least one 1.

Output

For each test case, output the final string if both players act optimally.

input
4 101 11001 0010 0101010000010100100101
output
1 101 00 01010000010100100101

In the first test case, Alice must delete the only 0. Bob may delete either occurrence of 1, so the resulting string is 1.

In the second test case, Alice may delete either occurrence of 0. Bob optimally deletes one of the first two occurrences of 1, so the resulting string is 101.

In the third test case, Alice may delete any occurrence of 0. Bob then deletes the only occurrence of 1, so the resulting string is 00.

B. Merge to Match

1.5 seconds, 256 megabytes

You are given two arrays a and b of lengths n and m , respectively. All $n + m$ integers in these arrays are **distinct**.

You can perform the following operation on a any number of times (possibly, zero):

- Choose two elements of a with values x and y , where $x \leq y$.
- Delete these two elements from a .
- Insert one integer z into a such that $x \leq z \leq y$.

After performing all operations, you may arrange the elements of a in any order.

Determine whether it is possible to make a equal to b .

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The first line of each test case contains two integers n and m ($1 \leq n, m \leq 2 \cdot 10^5$) — the lengths of the arrays a and b .

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$).

The third line contains m integers $b_1, b_2, \dots, b_m$ ($1 \leq b_i \leq 10^9$).

It is guaranteed that all $n + m$ integers in the arrays a and b are **distinct**.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

It is guaranteed that the sum of m over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print "YES" if it is possible to make a equal to b , and "NO" otherwise.

You can output the answer in any case (upper or lower). For example, the strings "yEs", "yes", "Yes", and "YES" will be recognized as positive responses.

input
11 2 1 1 3 2 3 2 1 3 5 2 4 4 2 1 3 5 7 4 6 4 2 2 5 8 11 1 10 4 2 1 4 7 9 3 10 5 2 10 1 100 6 4 90 5 6 3 1 4 10 20 30 40 3 15 35 4 2 1 2 3 100 4 5 7 3 1 8 3 30 18 12 25 2 15 28 4 1 1 2 3 5 4 4 2 1 3 5 6 2 4
output
YES NO YES NO NO YES YES NO YES YES YES YES

In the first test case, choose 1 and 3, delete them, and insert 2.

In the second test case, it can be shown that no sequence of operations can make a equal to b .

In the third test case, choose 1 and 5, delete them, and insert 4. Then, choose 3 and 7, delete them, and insert 6.

In the sixth test case, first choose 1 and 10, delete them, and insert 5. Next, choose 6 and 100, delete them, and insert 90. Finally, choose 4 and 5, delete them, and insert 5.

In the seventh test case, for the pairs (1, 4), (10, 20), and (30, 40), delete both elements and insert 3, 15, and 35, respectively.

In the ninth test case, delete the pairs (1, 3), (8, 18), and (25, 30) and insert 2, 15, and 28, respectively. Then, delete 12 and 15 and insert 15.

In the tenth test case, first choose 1 and 2, delete them, and insert 2. Then, choose 2 and 3, delete them, and insert 3. Finally, choose 3 and 5, delete them, and insert 4.

C. Maximize the Score

2 seconds, 256 megabytes

You are given an array a of length $2n$. Each integer from 1 to n occurs exactly twice in a .

Initially, your score is 0.

You can repeatedly perform the following operation while a is non-empty:

- Choose an integer x that is present in a .
- Let l and r be the indices of the leftmost and rightmost occurrences of x in the current array, respectively. If x occurs only once, then $l = r$.
- Add $(r - l + 1)^2$ to your score.
- Delete the elements $a_l, a_{l+1}, \dots, a_r$ from a . The remaining elements are concatenated without changing their order and re-indexed starting from 1.

Find the maximum possible score after making the array empty.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$).

The second line contains $2n$ integers $a_1, a_2, \dots, a_{2n}$ ($1 \leq a_i \leq n$).

It is guaranteed that each integer from 1 to n occurs exactly twice in a .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single integer — the maximum possible score.

input
6
1
1 1
2
1 2 1 2
2
1 2 2 1
3
1 1 2 3 3 2
3
1 2 3 3 2 1
4
1 2 3 4 1 2 3 4

output

4
10
16
20
36
28

In the second test case, one optimal strategy is to choose $x = 1$ first.

This deletes the subarray $[1, 2, 1]$ and adds $3^2 = 9$ to the score. The remaining array is $[2]$; choosing $x = 2$ adds 1. The total score is 10.

In the third test case, choosing $x = 1$ deletes the whole array and adds $4^2 = 16$ to the score.

In the fourth test case, choose $x = 1$ first and then choose $x = 2$. The total score is $2^2 + 4^2 = 20$.

In the sixth test case, choose $x = 2$ first. This deletes the subarray $[2, 3, 4, 1, 2]$ from the middle of the array and adds $5^2 = 25$ to the score. After deleting this subarray and concatenating the remaining elements, the array becomes $[1, 3, 4]$. Choosing each of the three remaining values then adds 1, so the total score is 28.

D. Good Pair Queries

2 seconds, 256 megabytes

You are given two binary strings s and t , both of length n .

For two binary strings a and b of the same length, the pair (a, b) is called *good* if both strings can be made empty by performing the following operation zero or more times:

- Choose a non-empty set of positions $1 \leq i_1 < i_2 < \dots < i_k \leq |a|$ and a character $c \in \{0, 1\}$.
- Let $x = a_{i_1} a_{i_2} \dots a_{i_k}$ and $y = b_{i_1} b_{i_2} \dots b_{i_k}$.
- The character c must be a mode* of both x and y .
- Delete the characters at the chosen positions from both strings. The remaining characters are concatenated without changing their relative order.

A binary string may have both 0 and 1 as modes.

You need to answer q queries. In each query, you are given two integers l and r . Determine whether the pair of substrings $(s_l s_{l+1} \dots s_r, t_l t_{l+1} \dots t_r)$ is good.

The queries are **independent**.

*For a binary string z , a character c is a mode if it appears at least $\left\lceil \frac{|z|}{2} \right\rceil$ times in z . Here, $\lceil x \rceil$ denotes the smallest integer greater than or equal to x .

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The first line of each test case contains two integers n and q ($1 \leq n, q \leq 2 \cdot 10^5$) — the length of each string and the number of queries.

The second line of each test case contains the binary string s of length n .

The third line of each test case contains the binary string t of length n .

Each of the next q lines contains two integers l and r ($1 \leq l \leq r \leq n$) — a query.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

It is guaranteed that the sum of q over all test cases does not exceed $2 \cdot 10^5$.

Output

For each query, output "YES" if the pair $(s_l s_{l+1} \dots s_r, t_l t_{l+1} \dots t_r)$ is good. Otherwise, output "NO".

You can output the answer in any case (upper or lower). For example, the strings "yEs", "yes", "Yes", and "YES" will be recognized as positive responses.

input
3 2 2 01 10 1 1 1 2 5 1 11111 11111 1 3 5 1 00011 01111 1 5
output
NO YES YES YES

In the first test case, for the first query, the pair is (0, 1). No character is a mode of both strings, so no operation can be performed.

For the second query, choose both positions and $c = 0$. Then $x = 01$ and $y = 10$, so c is a mode of both strings and the whole pair is deleted.

In the third test case, neither character is a mode of both 00011 and 01111, so the whole pair cannot be deleted in one operation. It can be emptied in three operations:

- Choose positions 2 and 4 and $c = 1$. Then $x = 01$ and $y = 11$, so c is a mode of both x and y . After deleting the selected positions, the remaining characters are concatenated, giving the pair (001, 011).
- Choose positions 1 and 2 and $c = 0$. Then $x = 00$ and $y = 01$, so c is a mode of both x and y . After deleting the selected positions, the pair becomes (1, 1).
- Choose the remaining position 1 and $c = 1$. Then $x = 1$ and $y = 1$, so c is a mode of both x and y . Deleting the selected position makes both strings empty.

E. Excuse for Breaks

2 seconds, 256 megabytes

You are given three integers n, m , and d , and two arrays $p_1, p_2, \dots, p_m$ and $r_1, r_2, \dots, r_m$. The array p is strictly increasing.

For a binary array a of any positive finite length (so $|a|$ need not equal n), define its value $f(a)$ using the following pseudocode:

```
function f(a):
    v := 0
    c := 0
    for i from 1 to length(a):
        if a[i] is equal to 1:
            v := v + d
            c := c + 1
        else:
            c := 0
    for j from 1 to m:
        if c is equal to p[j]:
            v := v + r[j]
        if c is equal to n:
            c := 0
    return v
```

Here, " $:=$ " denotes the assignment operation.

Let $I(a)$ denote the array $[1, 1, \dots, 1]$ of length $|a|$. In other words, $I(a)$ consists of $|a|$ ones.

Determine whether there exists a binary (consisting only of zeros and ones) array a such that $f(a) > f(I(a))$.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 2000$). The description of the test cases follows.

The first line of each test case contains three integers n, m , and d ($1 \leq n \leq 10^9, 0 \leq m \leq 2000, 0 \leq d \leq 10^9$).

The i -th of the next m lines contains two integers p_i and r_i ($1 \leq p_i \leq n, 1 \leq r_i \leq 10^9$).

The array p is strictly increasing.

It is guaranteed that the sum of m over all test cases does not exceed 2000.

Output

For each test case, output "YES" if such a binary array a exists, and "NO" otherwise.

You can output the answer in any case (upper or lower). For example, the strings "yEs", "yes", "Yes", and "YES" will be recognized as positive responses.

input
3 6 4 3 2 5 3 9 4 1 5 3 7 3 5 2 5 4 5 7 10 684492057 3 386217943 367971233 991739271 612599954 429216213 684492056 402931836
output
YES NO YES

In the first test case, you can choose $a = [1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1]$.

The array a contains 16 ones, so the total contribution of d is $16 \cdot 3 = 48$. Its runs of consecutive ones have lengths 9, 4, and 3. The corresponding total rewards are 32, 15, and 14, respectively. Therefore, $f(a) = 48 + 32 + 15 + 14 = 109$.

Moreover, $I(a)$ consists of 18 ones. During the computation of $f(I(a))$, they form three complete blocks of 6 ones, and each block contributes $6 \cdot 3 + 5 + 9 + 1 + 3 = 36$. Thus, $f(I(a)) = 3 \cdot 36 = 108$. Since $f(a) > f(I(a))$, the answer is "YES".

In the second test case, no binary array a satisfies $f(a) > f(I(a))$, so the answer is "NO".

F. Matrix Elimination

2 seconds, 256 megabytes

You are given an integer k and a matrix v with n rows and m columns. The element in row i and column j is denoted by $v_{i,j}$.

A cell (x, y) is called a *peak* if its value is greater than or equal to the sum of the values in all other cells in row x and column y . Formally, (x, y) is a peak if

$$v_{x,y} \geq \sum_{\substack{1 \leq i \leq n \\ i \neq x}} v_{i,y} + \sum_{\substack{1 \leq j \leq m \\ j \neq y}} v_{x,j}.$$

You may perform the following operation any number of times (possibly zero):

- Choose four integers x_l, x_r, y_l , and y_r ($1 \leq x_l \leq x_r \leq n$, $1 \leq y_l \leq y_r \leq m$).
- Subtract 1 from $v_{i,j}$ for every $x_l \leq i \leq x_r$ and $y_l \leq j \leq y_r$.

Find the minimum number of operations required to obtain a matrix with at least k peaks.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The first line of each test case contains three integers n, m , and k ($1 \leq n, m \leq 10^5, 1 \leq k \leq n \cdot m, n \cdot m \leq 10^5$) — the number of rows, the number of columns, and the required number of peaks.

The i -th of the next n lines contains m integers $v_{i,1}, v_{i,2}, \dots, v_{i,m}$ ($-10^9 \leq v_{i,j} \leq 10^9$).

It is guaranteed that the sum of $n \cdot m$ over all test cases does not exceed 10^5 .

Output

For each test case, output a single integer — the minimum number of operations required to obtain at least k peaks.

If there is no solution, print a single integer -1 .

```
input
16
3 3 7
-1 -30 7
6 -3 22
1 -18 16
3 1 2
100000000
300000000
200000000
1 3 1
2 3 4
1 3 1
1 2 3
1 2 2
5 7
5 2 8
531959596 -61416172
425363565 672913308
981527099 451180253
-161687136 898803495
-388356105 897723313
1 1 1
0
1 1 1
-5
1 4 3
-8 -4 4 -11
1 9 5
-12 -4 -13 9 -1 -15 6 -15 -4
1 3 3
-12 -15 8
1 5 5
10 8 -4 0 -4
1 3 3
12 9 -1
1 3 3
-14 11 -6
1 6 6
-1 0 15 3 1 -14
1 2 2
1000000000 -1000000000
```

```
output
2
100000000
1
0
2
734592698
0
-1
0
0
11
6
12
11
7
2000000000
```

For the first test case, you can perform the following two operations:

- choose $(x_l, x_r, y_l, y_r) = (1, 3, 2, 3)$;
- choose $(x_l, x_r, y_l, y_r) = (2, 3, 1, 3)$.

After these operations, the matrix will be:

-1	-31	6
5	-5	20
0	-20	14

The seven peaks are highlighted in green. For example:

- The cell $(1, 1)$ is a peak because $-1 \geq 5 + 0 - 31 + 6 = -20$;
- The cell $(3, 1)$ is a peak because $0 \geq -20 + 14 - 1 + 5 = -2$;
- The cell $(2, 1)$ is not a peak because $5 < -5 + 20 - 1 + 0 = 14$.

It can be shown that fewer than two operations cannot create seven peaks, so the answer is 2.

In the eighth test case, the only cell has value -5 . A cell in a 1×1 matrix is a peak if and only if its value is non-negative. Since every operation only decreases its value, it is impossible to make it a peak.

For the last test case, both cells are peaks exactly when their values are equal. Therefore, the first cell must be decreased from 10^9 to -10^9 , which requires $2 \cdot 10^9$ operations.

G. No Balance Left

6 seconds, 256 megabytes

Alisa has a shopping card with an initial balance of h . There are n types of products in a supermarket. A product of type i costs c_i , and Alisa can buy any number of products of each type.

There are also m rebate activities. The j -th activity applies to a purchase if its total cost is at least a_j and gives a rebate of b_j .

For each purchase, Alisa performs the following operations:

- She chooses one or more products whose total cost x does not exceed the current balance on her card and pays x using the card.
- Among all rebate activities that apply to this purchase, the one with the largest rebate takes effect and its rebate is added to the card balance. If no rebate activity applies, no rebate is added.

For every h from 1 to s , determine whether Alisa can make the balance on her card equal to 0 after finitely many purchases.

Input

The first line contains three integers n, m , and s ($1 \leq n, m, s \leq 125\,000$) — the number of product types, the number of rebate activities, and the maximum initial balance to consider, respectively.

The second line contains n integers $c_1, c_2, \dots, c_n$ ($1 \leq c_i \leq 125\,000$) — the price of a product of each type.

- She buys one product of the first type again for 4. No rebate activity applies, so the balance decreases from 4 to 0.

Therefore, the 15-th line of the first example output is "YES".

In the second example, if the initial balance is 4, Alisa can buy one product of the first type for 4. Since $4 < 6$, no rebate activity applies, and the balance becomes 0. Therefore, the fourth line of the second example output is "YES". If the initial balance is 1, no product is affordable, so the first line is "NO".

You can output the answer in any case (upper or lower). For example, the strings "yEs", "yes", "Yes", and "YES" will be recognized as positive responses.

[illegible]

- She buys one product of each type for a total cost of $4 + 7 = 11$. The balance decreases from 15 to 4.
- Since $11 \geq 8$, the rebate of 4 takes effect. The balance increases from 4 to 8.
- She buys one product of the first type for 4. No rebate activity applies, so the balance decreases from 8 to 4.

[Codeforces](#) (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform